

Proving Properties of Compiled Daml Packages

A guided introduction to the verification pipeline

Daml Access Graph

Abstract

This document explains how the verification pipeline in this repository proves properties of a compiled Daml package, and, just as importantly, what its results do not mean. It is written to be read start to finish by someone who knows Daml but not formal verification. Section 2 carries a single example the whole way through, from a Daml choice to a solver verdict, before any general theory appears. The formal companion to this document is `formal/SPECIFICATION.md` (definitions and theorems, stated tersely) and `formal/Correspondence.md` (the trust audit).

Contents

1	The problem	2
1.1	The correspondence gap	2
1.2	The pipeline	2
2	A complete worked example	3
2.1	The source	3
2.2	The extracted transition	3
2.3	The property	3
2.4	The verification condition	3
2.5	Where the theorem enters	4
3	The intermediate representation	4
3.1	Syntax	4
3.2	Semantics	4
4	Transitions and properties	5
5	Generating the verification condition	5
6	Soundness	5
7	Dropping guards	6
8	Closing the correspondence gap	6
9	From one transition to a whole ledger	7
10	Reading a verdict	7
10.1	The asymmetry	7
10.2	A worked triage	8
11	Limits	8

1 The problem

Suppose we want to know that a token package never loses value: whenever a choice archives a contract and creates replacements, the amounts add up. A test suite can sample this. It cannot establish it, because a choice takes arguments from an infinite domain and a test tries finitely many.

The alternative is to prove it. That raises a question which is easy to state and easy to get wrong.

1.1 The correspondence gap

A proof is always a proof *about some model*. If we write a symbolic model of a contract by hand and verify the model, we have established

$$M \models \varphi$$

for our hand-built M . Whether M faithfully describes the contract C that is actually deployed is a separate claim,

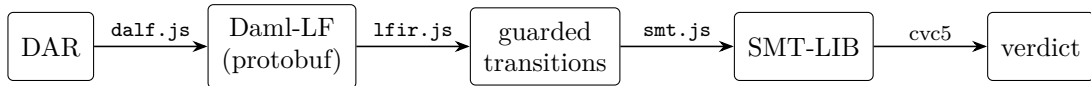
$$\llbracket C \rrbracket = M,$$

and in most developments that claim is an unproven comment. It is also where the interesting bugs live: a model drifts from its implementation silently, and a verified model of the wrong thing is worse than no model at all, because it carries the authority of a proof.

Intuition. Verifying a hand-written model answers “is my description of the contract correct?” We want an answer to “is the contract correct?” The two coincide only if the description matches, and that match is exactly what a hand-written model leaves unchecked.

This pipeline attacks the gap directly by deriving the model from the compiled artifact. Nothing is transcribed by hand. The DAR that would be deployed is the input, and the transition system is extracted from it. For the fragment the translator handles, the correspondence holds by construction; where the translator cannot proceed, it says so in the output rather than guessing.

1.2 The pipeline



The claim proved in Lean, for the fragment of Section 8, is

$$\boxed{\models \text{VCGen}(P, \varphi) \implies P \models \varphi} \quad (1)$$

read as: if the generated verification condition is valid, the package really does satisfy the property. The solver establishes the left-hand side. The theorem is what licenses the step to the right-hand side.

2 A complete worked example

Before any general definitions, here is the entire pipeline on one choice. The example is a *split*: a contract holding an amount is archived, and two contracts are created holding x and the remainder.

2.1 The source

```
choice Split : (ContractId Piece, ContractId Piece)
  with x : Decimal
  controller owner
  do
    a <- create Piece with amount = x
    b <- create Piece with amount = this.amount - x
    pure (a, b)
```

The property we want is that the split conserves value: the two pieces together hold exactly what the archived contract held.

2.2 The extracted transition

The translator reduces the choice to the data the property needs: the guards that must hold, the amounts assigned by each `create`, and the amount of the contract being archived. Variables are named by the projection path that produced them, so `arg.x` is the choice argument and `this.amount` is a field of the exercised contract.

$$\tau = \left\langle \underbrace{[]}_{\text{guards}}, \underbrace{[\text{arg.x}, \text{this.amount} - \text{arg.x}]}_{\text{created amounts}}, \underbrace{\text{this.amount}}_{\text{archived}} \right\rangle$$

This is `splitTransition` in `formal/Formal/Ledger.lean`, where it serves the same illustrative purpose.

2.3 The property

Conservation says: in every environment where the guards hold, the created amounts sum to the archived amount. There are no guards here, so the condition is unconditional:

$$\forall \rho. \llbracket \text{arg.x} \rrbracket_{\rho} + (\llbracket \text{this.amount} \rrbracket_{\rho} - \llbracket \text{arg.x} \rrbracket_{\rho}) = \llbracket \text{this.amount} \rrbracket_{\rho}$$

Here ρ is an *environment*: an assignment of a rational number to every variable name. Quantifying over all of them is what makes the result a proof rather than a sample.

2.4 The verification condition

We do not ask the solver “is this true?” Solvers answer satisfiability questions, so we ask the dual: *is there any environment making the property false?* We assert the guards, assert the negation of the goal, and ask whether the set has a model.

```
(declare-fun |arg.x| () Real)
(declare-fun |this.amount| () Real)
(assert (not (= (+ |arg.x| (- |this.amount| |arg.x|)) |this.amount|)))
(check-sat)
```

`cvc5` answers `unsat`: no such environment exists. No counterexample exists, so the property holds in every environment, and the verdict is `PROVED`.

Intuition. `unsat` is the good answer. It means the search for a counterexample was exhaustive and came back empty. A `sat` answer hands back a concrete environment, which is why counterexamples can be printed but proofs cannot.

2.5 Where the theorem enters

The solver proved a fact about a *formula*. We want a fact about a *choice*. Three steps separate them, and each is discharged differently:

Step	Justified by	Status
choice $\rightarrow$ transition	<code>lfir.js</code>	tested; proved on a fragment
transition $\rightarrow$ formula	VCGen, Theorem 6.1	machine-checked
formula $\rightarrow$ <code>unsat</code>	<code>cvc5</code>	trusted

The middle row is the one this development contributes. The rest of the document makes it precise, then returns to the outer two rows, which are where the residual risk lives.

3 The intermediate representation

3.1 Syntax

Terms are indexed by a sort, either real or boolean.

Definition 3.1 (Sorts and terms). $s ::= \mathbb{R} \mid \mathbb{B}$, and terms $t : \text{Term } s$ are generated by

$$\begin{aligned}
t^{\mathbb{R}} &::= q \mid x^{\mathbb{R}} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \cdot t_2 \mid t_1 / t_2 \mid t_1 \text{ div } t_2 \mid t_1 \bmod t_2 \\
t^{\mathbb{B}} &::= b \mid x^{\mathbb{B}} \mid t_1 < t_2 \mid t_1 \leq t_2 \mid t_1 > t_2 \mid t_1 \geq t_2 \\
&\mid t_1 =_{\mathbb{R}} t_2 \mid t_1 =_{\mathbb{B}} t_2 \mid \neg t \mid t_1 \wedge t_2 \mid t_1 \vee t_2
\end{aligned}$$

with $q \in \mathbb{Q}$ and $b \in \{\text{true}, \text{false}\}$.

Indexing terms by their sort is a deliberate design choice with a payoff. An ill-sorted term such as “true + 1” cannot be written down at all, so evaluation needs no error case and every later theorem avoids side conditions about undefined terms. The JavaScript emitter achieves the same discipline dynamically, by rejecting ill-sorted queries in `inferSorts`.

3.2 Semantics

Definition 3.2 (Environment and evaluation). An environment $\rho = (\rho_{\mathbb{R}}, \rho_{\mathbb{B}})$ is a pair of total maps from names to $\mathbb{Q}$ and to $\mathbb{B}$. Evaluation $\llbracket \cdot \rrbracket_{\rho}$ is defined by structural recursion, homomorphically on each operator:

$$\llbracket q \rrbracket_{\rho} = q, \quad \llbracket x^s \rrbracket_{\rho} = \rho_s(x), \quad \llbracket t_1 + t_2 \rrbracket_{\rho} = \llbracket t_1 \rrbracket_{\rho} + \llbracket t_2 \rrbracket_{\rho},$$

and correspondingly for the remaining constructors.

Two modelling decisions deserve comment.

Exact rationals. The carrier is $\mathbb{Q}$, not floating point. Daml’s `Numeric 10` is modelled as an exact real. This is sound for sign, for ordering, and for division safety, and *unsound* for exact equalities on paths that round. The pipeline therefore refuses equality properties on transitions carrying rounding builtins rather than proving them wrongly.

Total division. Division is total, with $t/0 = 0$, following SMT-LIB. This looks alarming and is harmless, because division safety is discharged as a separate obligation (Section 4) instead of being assumed. A proof that never relies on the value of $t/0$ is unaffected by the convention chosen for it.

4 Transitions and properties

Definition 4.1 (Guarded transition). A transition is $\tau = \langle G, A, a_{\text{this}} \rangle$ where G is a list of boolean guards, A a list of created amounts, and a_{this} the archived contract's amount.

G contains the *usable* guards. A guard the translator could not render is dropped, and Section 7 explains why that is safe for proving.

Write Σ for the sum that matches the emitted term shape:

$$\Sigma([]) = 0, \quad \Sigma([a]) = a, \quad \Sigma(a :: r) = a + \Sigma(r).$$

Definition 4.2 (Amount conservation).

$$\text{Conserves}(\tau) \equiv \forall \rho. \left(\forall g \in G. \llbracket g \rrbracket_\rho = \text{true} \right) \implies \Sigma(\llbracket a \rrbracket_\rho : a \in A) = \llbracket a_{\text{this}} \rrbracket_\rho$$

Definition 4.3 (Division safety). For $\delta = \langle G, D \rangle$ with D the denominators occurring on the transition,

$$\text{DivisionSafe}(\delta) \equiv \forall \rho. \left(\forall g \in G. \llbracket g \rrbracket_\rho = \text{true} \right) \implies \forall d \in D. \llbracket d \rrbracket_\rho \neq 0.$$

Both are universally quantified over environments. That quantifier is the entire difference between a proof and a test.

5 Generating the verification condition

A solver is a satisfiability engine, so the property must be posed as a search for a counterexample. The pipeline asserts the guards $g_1, \dots, g_n$ and the negated goal, then calls **check-sat**. An **unsat** answer on that set is equivalent to validity of a single formula, and that formula is what VCGen constructs. Writing $\bigwedge(G, \text{tail})$ for the right fold $g_1 \wedge (g_2 \wedge (\dots \wedge \text{tail}))$:

Definition 5.1 (Verification conditions).

$$\begin{aligned} \text{VCGen}(\tau) &= \neg \left(\bigwedge(G, \neg(\Sigma(A) =_{\mathbb{R}} a_{\text{this}})) \right) \\ \text{VCGen}_{\div}(\delta) &= \neg \left(\bigwedge(G, \neg \bigwedge_{d \in D} (d \neq 0)) \right) \end{aligned}$$

Intuition. Read VCGen as “it is not the case that the guards hold while the goal fails”. That is precisely the sentence a **unsat** answer certifies, which is why the generator is shaped this way rather than as a direct implication.

6 Soundness

The generator would be worthless if a valid VC failed to entail the property. It does entail it, and the converse holds too.

Theorem 6.1 (Soundness, **vcgen_sound**). $(\forall \rho. \llbracket \text{VCGen}(\tau) \rrbracket_\rho = \text{true}) \implies \text{Conserves}(\tau).$

Theorem 6.2 (Completeness, `vcgen_complete`). $\text{Conserves}(\tau) \implies (\forall \rho. \llbracket \text{VCGen}(\tau) \rrbracket_\rho = \text{true})$.

Theorem 6.3 (`vcgen_iff`). $(\forall \rho. \llbracket \text{VCGen}(\tau) \rrbracket_\rho = \text{true}) \iff \text{Conserves}(\tau)$.

The same three hold for division safety as `vcgenDiv_sound`, `vcgenDiv_complete` and `vcgenDiv_iff`. All are proved in Lean 4 with no `sorry` and no user axioms; `#print axioms` reports only `propext` and `Quot.sound`, both kernel axioms.

Caution. Completeness is a statement about VCGen, not about the solver. It says the formula faithfully captures the property, so nothing is lost in translation. It does not say `cvc5` will decide it: the solver may still return `unknown` or exhaust its budget on a valid VC.

7 Dropping guards

Real packages contain guards the translator cannot render, typically text manipulation that has no counterpart in the arithmetic fragment. The pipeline drops such a guard and records that it did. This is safe in one direction only, and the asymmetry is worth understanding because it explains much of the output.

Theorem 7.1 (`drop_guards_sound`). *If $U \subseteq F$, then*

$$\text{Conserves}(\langle U, A, a_{\text{this}} \rangle) \implies \text{Conserves}(\langle F, A, a_{\text{this}} \rangle).$$

Intuition. Guards are assumptions. Proving a property while assuming *less* gives a *stronger* result. If the amounts balance for every environment whatsoever, they certainly balance for the environments the real guards permit. So a dropped guard can never turn a false statement into a proved one.

The cost is completeness. With fewer assumptions the solver explores states the real code cannot reach, and it may return a counterexample living in exactly those states. This is why a `DISPROVED` whose counterexample might have been excluded by a dropped guard says so explicitly in the report. The analogous result for the per-denominator coverage split is `divsafe_denominators_mono`, which is what allows a partial verdict naming which obligations were discharged.

8 Closing the correspondence gap

Everything so far concerns the IR. None of it says the IR faithfully represents the compiled LF, which is the gap of Section 1 reappearing one level down. For a fragment, that arrow is closed too.

`formal/Formal/MiniLF.lean` defines a sort-indexed fragment of post-inlining Daml-LF with its own semantics $\llbracket \cdot \rrbracket^{\text{LF}}$, including projections and let-binding. `Translate.lean` mirrors the JavaScript translation as a function $\mathcal{T}$, and the correctness theorem relates the two:

Theorem 8.1 (`translate_correct`). *If $\text{Agrees}(\rho, \Gamma_T, \Gamma_L)$, then $\llbracket \mathcal{T}_{\Gamma_T}(e) \rrbracket_\rho = \llbracket e \rrbracket_{\rho, \Gamma_L}^{\text{LF}}$.*

Here `Agrees` relates the translator’s binding environment to the one the source semantics uses. Composing with Theorem 6.1 yields the end-to-end statement:

Theorem 8.2 (`minilf_vcgen_sound`). $(\forall \rho. \llbracket \text{VCGen}(\mathcal{T}(t)) \rrbracket_\rho = \text{true}) \implies \text{ConservesLF}(t)$.

The conclusion is stated purely in source-fragment semantics. For a transition inside MiniLF, an `unsat` from `cvc5` is a proof about the *source expression*, not merely about the IR the translator happened to build.

Outside MiniLF, the claim that a given LF expression denotes a given IR term remains untrusted JavaScript. That residue is enumerated explicitly in `formal/Correspondence.md` rather than left implicit.

9 From one transition to a whole ledger

A property proved of each choice in isolation is not yet a property of a ledger. Lifting it requires an induction, supplied in `formal/Formal/Ledger.lean`.

Let $\mathcal{S}$ be a ledger state, Step the relation induced by a set of transitions, Reachable its reflexive transitive closure, and $\text{total}_f(\mathcal{S})$ the sum of field f over active contracts.

Theorem 9.1 (`conservation_global`). *If the verification condition is valid for every transition lt in the package, then for all $\mathcal{S}$ with $\text{Reachable}(\mathcal{S}_0, \mathcal{S})$,*

$$\text{total}_f(\mathcal{S}) = \text{total}_f(\mathcal{S}_0).$$

The proof is by induction over reachability, from `invariant_reachable` and `step_preserves_total`. The companion theorem `no_template_reachable` gives template-level invariants by the same route.

This layer is generic in the numeric carrier: `Model.lean` abstracts $\mathbb{Q}$ to any `RealModel`, and the algebraic laws the induction needs are *hypotheses of the theorems that use them*, not axioms. To rule out the reading “the theorem holds because its hypotheses are unsatisfiable”, the development instantiates the entire chain on the split transition of Section 2 and derives the global invariant concretely.

Caution. What this does not establish is that the transitions the translator extracts are the steps a real Canton ledger can take. That is the untrusted residue of Section 8, plus the fidelity of the ledger abstraction itself.

10 Reading a verdict

Verdict	Solver	Meaning
PROVED	unsat	No counterexample exists in the model.
PROVED-BOUNDED	unsat	As above, up to the stated list-unrolling bound.
PROVED-PARTIAL	unsat	As above, for a stated subset of obligations.
DISPROVED	sat	A concrete counterexample, printed.
NOT-MODELLABLE	not run	Refused, with a machine-generated reason.
NOT-APPLICABLE	not run	The property says nothing here.
SOLVER-UNKNOWN	unknown	No claim.

10.1 The asymmetry

`unsat` and `sat` do not carry equal weight, and conflating them is the most common way to misread this tool.

An `unsat` is exhaustive: it certifies that no environment falsifies the property. A `sat` produces one environment, and that environment may be an artifact of the modelling rather than a behaviour of the code. Two mechanisms produce such artifacts:

1. A **dropped guard** (Section 7) may have excluded the counterexample in the real system.
2. An **uninterpreted function** may have been given an interpretation the real function never takes. Text operations are modelled this way, since they have no counterpart in the arithmetic fragment.

Every verdict resting on either discloses it, and a `DISPROVED` carries an explicit caveat naming the symbols involved.

Intuition. PROVED is a theorem. DISPROVED is a lead. Triaging a DISPROVED means checking whether its counterexample survives the caveats, which is a human judgement the tool deliberately does not make.

10.2 A worked triage

Two sibling templates in one package each guard an amount, and both report DISPROVED for non-negativity. The verdicts look identical and mean opposite things.

The first carries a dropped guard: its real check is a comparison the translator could not render, so the solver never saw it. Inspecting the source shows the check is present and correct, and the finding is an artifact.

The second carries no dropped guard, because its check *was* translated faithfully. The check reads

```
validateAmount : Decimal -> Bool
validateAmount _amount = True
```

against a sibling implementation of `amount >= 0.0`. The predicate is vacuous, so every call site that appears to enforce non-negativity enforces nothing, including one whose failure branch reads `abort "amount cannot be negative"`. The finding is real.

The discriminator was not the verdict. It was the disclosure attached to it.

11 Limits

Stated plainly, because a document that hides its limits is worse than none.

- **damlc is trusted.** Daml source to DAR is outside the development; the compiler defines what the package means.
- **cvc5 is trusted.** An unsat is taken at its word. Checking solver-emitted proof certificates is future work.
- **The protobuf reader is tested, not verified.** A misread yields a wrong IR, and the theorems begin at the IR, so they cannot detect it.
- **The translator is verified only on MiniLF.** It is the trust root of this pipeline, as `damlc` is the trust root of the ledger.
- **Numeric 10 is exact.** Sound for sign, order and division safety; unsound for exact equalities on rounding paths, which are refused rather than proved.
- **The Int sort is not in the Lean development.** The emitter distinguishes Daml `Int` from `Numeric` and asserts integrality, but Lean's sorts remain $\{\mathbb{R}, \mathbb{B}\}$, so every theorem above is stated over the two-sort fragment. This gap deserves particular attention: integrality *shrinks* the model class, so a misclassified field could in principle yield a false PROVED. The emitter mitigates this by deriving integrality only from the compiled type declaration, never by inference, but that argument is presently a code invariant and a test rather than a theorem.

A Notation

Symbol	Meaning
$\llbracket t \rrbracket_\rho$	value of term t in environment ρ
$\models \varphi$	φ evaluates to true in every environment
$\tau = \langle G, A, a \rangle$	guarded transition
Σ	sum matching the emitted term shape
$\text{VCGen}, \text{VCGen}_\div$	verification-condition generators
unsat, sat	solver answers
$\mathcal{T}$	the translator, LF to IR

B Where to look

Concept	Lean	JavaScript
Sorts, terms	<code>Term.lean</code>	<code>lfir.js</code>
Semantics	<code>Eval.lean</code>	<code>ir-eval.js</code>
Transitions	<code>Transition.lean</code>	<code>lfir.js</code>
VC generation	<code>VCGen.lean</code>	<code>smt.js</code>
Soundness	<code>Soundness.lean</code>	(none)
Division safety	<code>Division.lean</code>	<code>smt.js</code>
Source fragment	<code>MiniLF*.lean</code>	<code>lfir.js</code>
Ledger induction	<code>Ledger.lean</code>	(none)

Build the Lean development with `lake build` in `formal/`. It uses core Lean 4 only, with no `mathlib` dependency.